

Task 1: Firmware Foundations & Environment Setup

Deadline: 4 days

Applies to: All enrolled participants (VSDSQ Mini board not required)

Objective of Task 1

The goal of Task 1 is to ensure that every participant:

- understands **what firmware libraries are**
- understands **how a development board, firmware, and APIs connect**
- has a **working development environment** ready before the board arrives
- is aligned on **what will be built during the 4-week internship**

This task does **not** require the physical board.

What this internship is about (important to read)

This internship focuses on **embedded firmware development on a RISC-V SoC**.

Participants will:

- work in **Embedded C**
- interact with **hardware registers through APIs**
- build **core firmware utilities** (GPIO, timing, serial)
- develop **one complete peripheral driver/library**

This is **not** an RTL / Verilog / chip design program.

Task 1 – What you need to do

Task 1 has **three parts**.

All three must be completed.

Part A: Understand firmware libraries (conceptual)

Reading (mandatory)

Read and understand the following concepts:

1. **What is embedded firmware?**
 - Difference between application code and firmware
 - Role of firmware in microcontroller-based systems
2. **What is a firmware library / API?**
 - Why functions like `digitalWrite()`, `delay()`, `Serial.write()` exist

- How libraries hide register-level complexity
- Why reusable APIs are critical in industry

3. Bare-metal firmware basics

- What “bare-metal” means
- Why no RTOS is used in this program
- How timing, polling, and interrupts are handled without an OS

You do NOT need prior experience — just clarity of concepts.

Part B: Development environment setup (hands-on)

This internship uses **standard embedded development tools**.

You must install them on your system.

Supported OS

- Windows 10 / 11
- macOS
- Linux (Ubuntu preferred)

Step 1: Install Git

Git is required to clone repositories and submit work.

- Windows: <https://git-scm.com/download/win>
- macOS: `brew install git` (or Xcode Command Line Tools)
- Linux: `sudo apt install git`

Verify installation:

```
git --version
```

Step 2: Install a C compiler (GCC)

For Task 1, **host GCC** is sufficient (RISC-V GCC will be introduced later).

- Windows: Install via MSYS2 or MinGW
- macOS: Xcode Command Line Tools
- Linux: `sudo apt install build-essential`

Verify:

```
gcc --version
```

Step 3: Install a code editor

Use **any one**:

- VS Code (recommended)
- Sublime Text
- Vim / Emacs

No IDE is mandatory.

Step 4: Clone the internship repository

(Repo link will be shared separately)

```
git clone https://github.com/vsdip/vsdsquadron-mini-core
cd vsdsquadron-mini-core
```

Part C: Lab – Understanding a simple firmware library

This lab is **board-independent**.

What you will do

- Study a **simple firmware-style C library**
- Understand how APIs are structured
- Build and run it on your system

Lab structure (example)

You will find:

```
/task1
├── main.c
├── gpio.c
├── gpio.h
└── README.md
```

What to observe

- `gpio.h` defines the API (function declarations)
- `gpio.c` contains the implementation
- `main.c` uses the API without knowing internal details

This mirrors **real embedded firmware design**.

Build and run

```
gcc main.c gpio.c -o task1_demo
./task1_demo
```

Expected output:

- Console logs showing simulated GPIO behavior

What to submit (Images mandatory)

Each participant must submit:

1. A **short written explanation WITH COMPULSORY IMAGES** for each of below item (1–2 pages or Markdown):
 - What is a firmware library?
 - Why APIs are important in embedded systems?

- What was understood from the lab code?
- 2. A **screenshot (with your username clearly displayed on terminal)** showing:
 - Successful compilation
 - Program output
- 3. GitHub repository link (personal) showing all of the above